

Set up your coding assistant with Gemini MCP and Skills | Gemini API

 来源: <https://ai.google.dev/gemini-api/docs/coding-agents>

Try the new high-speed, cost-effective [Veo 3.1 Lite](#) model for video generation at scale.

AI coding assistants are powerful but have limitations—training data cuts off at a specific date, missing new API features and changes. Without access to Gemini-specific documentation, agents may suggest generic patterns instead of optimized approaches.

To keep your coding assistant current with the evolving Gemini API and its recommended usage, we recommend setting up the **Gemini Docs MCP** and enhancing your environment with **Gemini API Skills**. While these tools are usable independently, they are designed to work together to provide complete coverage.

Connect the Gemini Docs MCP

Gemini hosts a public Model Context Protocol (MCP) server at `https://gemini-api-docs-mcp.dev`. Connecting your coding agent to this server ensures that all queries have access to the latest APIs, code updates, and optimal configuration examples.

Run the following command in your agent's terminal or project root to install the server:

```
npx add-mcp "https://gemini-api-docs-mcp.dev"
```

This server adds a `search_documentation` function that your agent can use to retrieve real-time API definitions and integration patterns from the official Gemini documentation files.

Add API Development Skills

The skills provide **baked-in rules and best practices** (such as enforcing the correct SDK and current model versions) directly in your assistant's context. The skill works together with the Gemini Docs MCP service: If you have both installed, the skill uses the MCP service for documentation, but even without the MCP installed, it will fetch `llms.txt`

from `ai.google.dev` as a fallback.

To install these skills, you can use one of the following supported tools. Installation instructions for both are provided below each skill module:

- [skills.sh](#): Recommended. The open standard for portable agent behaviors.
- [Context7](#): Supported for users already utilizing the Context7 ecosystem.

gemini-api-dev

The foundational skill for general-purpose Gemini development. This skill provides documentation and best practices for:

- Prompt routing to current models (e.g., Gemini 3.1 Pro/Flash) and avoiding deprecated models
- Multimodal prompting, function calling, structured outputs, and common integration patterns

Install with skills.sh

```
npx skills add google-gemini/gemini-skills --skill gemini-api-dev -  
-global
```

Install with Context7

```
npx ctx7 skills install /google-gemini/gemini-skills gemini-api-dev
```

gemini-live-api-dev

Skill for building real-time conversational AI applications with Gemini Live API. This skill provides documentation and best practices for:

- WebSocket connections for low-latency streaming
- Streaming audio, video, and text
- Voice activity detection and barge-in support

Install with skills.sh

```
npx skills add google-gemini/gemini-skills --skill gemini-live-api-dev --global
```

Install with Context7

```
npx ctx7 skills install /google-gemini/gemini-skills gemini-live-api-dev
```

Skill for building apps with the [Interactions API](#). The Interactions API is a unified interface for interacting with Gemini models and agents, designed for agentic applications. This skill covers:

- Text generation, multi-turn chat, and streaming
- Function calling, structured output, and image generation
- Background execution and Deep Research agents
- Server-side conversation state management
- Python and TypeScript SDK patterns

```
npx skills add google-gemini/gemini-skills --skill gemini-interactions-api --global
```

```
npx ctx7 skills install /google-gemini/gemini-skills gemini-interactions-api
```

Verify installation

After installing, confirm that your coding assistant can connect to the Gemini Docs MCP server and use your installed skills.

1. Verify agent behavior

The most reliable way to verify is to ask your agent a technical question about Gemini API.

Prompt: "How do I use context caching with the Gemini API?"

A successful setup will:

- **Provide accurate code:** Reference specific Gemini methods like `cacheContent` or `cachedContents.create` from the latest endpoints.
- **Use the MCP Tool:** Show that it is connected to the **Gemini Docs MCP Server** or utilizing the `search_documentation` tool to fetch data.
- **Invoke loaded skills:** Show an indicator that it is "Using skill: gemini-api-dev" (if relying on a secondary wrapper).

2. Verify manifestations & tools

If the agent gives a general or generic answer, use the specific Discovery or Status commands for your environment to verify that the Docs MCP or skill is loaded into memory.

ENVIRONMENT	MCP VERIFICATION	SKILLS VERIFICATION
Claude Code	Type <code>/mcp</code> in the terminal to view active servers and <code>search_documentation</code> tools.	Type <code>/skills</code> in the terminal to list all active manifests.
Cursor	Navigate to Settings > Features > MCP . Ensure server is "Connected".	Open Settings > Rules . Verify the skill appears under "Agent Decides."
Antigravity	Check the Customizations > Connections sidebar for MCP status.	Type <code>/skills list</code> or check the Customizations > Rules sidebar.
Gemini CLI	Run <code>gemini mcp list</code> or use <code>/mcp list</code> .	Run <code>gemini skills list</code> or use the <code>/skills slash</code> command in-session.
Copilot	Type <code>@gemini /mcp</code> to list active data connectors.	Type <code>@gemini /skills</code> (or <code>/skills</code>) to view active extensions.

Troubleshooting

If your agent provides only general information or fails to recognize Gemini-specific methods, check the following:

Agent didn't discover the skill

Most agents index skills only on startup.

Fix: Completely restart your IDE (Cursor/VS Code) or exit and re-open your terminal-based agent (Claude Code).

Global vs. local conflict

If you installed with the `--global` flag, your agent might be ignoring it in favor of project-specific rules.

Fix: Try installing the skill directly into your project root without the global flag:

```
npx skills add google-gemini/gemini-skills --skill gemini-api-dev
```